

Day-70-Weekly Revision Git and Data Collection

1. Day Number

Day 70

2. Topic Name

Weekly Revision — Git and Data Collection

Today you will revise **Days 64–69**:

- Git and GitHub fundamentals
 - CSV files
 - Excel files
 - JSON data
 - API requests
 - Basic debugging and error handling
-

3. Connection

Over the last few days, you learned two important developer skills.

First, you learned how to **track your Python project with Git**.

Then you learned how Python can collect and work with **structured external data** such as CSV, Excel, JSON, and API responses.

Today you will combine those ideas in one very small project.

The basic workflow is:

```
Create project
  ↓
Load structured data
  ↓
Handle a possible error
  ↓
Print useful values
  ↓
Track the project with Git
```

4. Revision Summary of Days 64–69

Day 64 — Git and GitHub Fundamentals

Git helps you track changes in a project.

Important commands included:

```
git init
git status
git add main.py
git commit -m "First commit"
```

A simple Git workflow is:

```
change file
  ↓
git status
  ↓
git add
  ↓
git commit
```

Git works locally on your computer.

GitHub can store your Git repository online.

Day 65 — CSV Files

CSV stores tabular data using rows and columns.

Example:

```
name,price
Keyboard,1200
Mouse,600
```

Python provides the built-in:

```
csv
```

module for working with CSV files.

Day 66 — Excel Files

Excel also stores rows and columns, but Excel files can contain:

```
Workbook
├── Worksheet
│   ├── Rows
│   ├── Columns
│   └── Cells
```

Unlike CSV, working with `.xlsx` files normally requires an external library such as:

```
openpyxl
```

Day 67 — JSON Data

JSON stores structured information using keys and values.

Example:

```
{  
    "name": "Asha",  
    "city": "Bengaluru"  
}
```

Python's built-in `json` module can read and write JSON.

Important functions:

```
json.load()  
json.dump()
```

Day 68 — API Data Collection

An API allows one program to request data from another system.

A simple API flow looks like:

```
Python program  
    ↓  
GET request  
    ↓  
Server  
    ↓  
JSON response  
    ↓  
Python program
```

You also learned that HTTP status codes can help determine whether the request succeeded.

Day 69 — Debugging and Error Handling

External data can fail even when your Python code is written correctly.

For example:

```
file does not exist  
JSON contains invalid data
```

```
internet connection fails
API server is unavailable
```

Basic tools included:

```
try
except
print()
```

The goal is not only to prevent crashes.

The goal is to make the program fail **gracefully and understandably**.

5. Important Topics

Today's main revision topics are:

```
Git
CSV
Excel
JSON
API requests
try / except
file handling
structured data
functions
debugging
```

You do not need to use all of them in today's program.

The exercise should remain small.

6. Foundational Notes

Structured Data

Structured data follows an organized format.

For example:

```
name,city
Asha,Bengaluru
Rahul,Delhi
```

or:

```
[
  {
```

```
[
    {
        "name": "Asha",
        "city": "Bengaluru"
    },
    {
        "name": "Rahul",
        "city": "Delhi"
    }
]
```

Both examples contain similar information.

The storage format is different.

External Data Can Fail

Suppose your program contains:

```
open("customers.json")
```

The Python syntax might be perfectly correct.

But the program can still fail if:

```
customers.json does not exist
```

This is why data programs often need error handling.

Git Does Not Run Your Program

Git and Python have different jobs.

```
Python
→ runs your program

Git
→ tracks changes to your project
```

For example:

```
main.py changes
↓
Git notices the change
↓
You add the change
↓
You create a commit
```

7. Easy Example

Imagine this small file.

products.json

```
[
  {
    "name": "Keyboard",
    "price": 1200
  },
  {
    "name": "Mouse",
    "price": 600
  }
]
```

Your Python program could conceptually:

```
open products.json
  ↓
convert JSON into Python data
  ↓
read each product
  ↓
print product name and price
```

Possible output:

```
Keyboard - 1200
Mouse - 600
```

But what happens if `products.json` does not exist?

Instead of showing a large error traceback, your program could display something simple such as:

```
Data file could not be found.
```

That is where basic error handling becomes useful.

8. Revision Problem Statement

Create a **tiny Python project** that reads a small **JSON or CSV dataset** and prints a few values.

Your project should also:

- use a function to load or process the data
- handle one simple data-loading error
- avoid crashing when the data file is missing
- initialize Git in the project
- add the project files to Git
- create a commit

For example, your project might look like:

```
product_project/  
├── main.py  
└── products.json
```

or:

```
product_project/  
├── main.py  
└── products.csv
```

Keep the dataset very small—around **2 or 3 records**.

9. Concepts Used

For this revision exercise, you may use:

```
import json
```

or:

```
import csv
```

along with:

```
def  
open()  
try  
except  
print()  
for
```

Git concepts:

```
repository  
status
```

```
staging
commit
```

You do **not** need APIs or Excel in the final exercise.

They are part of today's revision, but the practice program should remain small.

10. Thought Process

Before writing Python code, think through the problem.

Step 1: What data will I store?

For example:

```
product name
product price
```

Step 2: Which format will I use?

Choose only one:

```
CSV
```

or:

```
JSON
```

JSON may look like:

```
[
  {"name": "Keyboard", "price": 1200},
  {"name": "Mouse", "price": 600}
]
```

Step 3: What should my function do?

You could create a function such as:

```
load_products()
```

Its responsibility could be:

```
open file
read data
return data
```

Step 4: What could fail?

The easiest failure to handle today is:

```
file does not exist
```

Think about where:

```
try
```

and:

```
except
```

should be placed.

Step 5: What should happen after loading?

If the data loads successfully:

```
loop through records  
print a few values
```

Step 6: How will Git track the project?

After creating the files:

```
git init
```

Check them:

```
git status
```

Stage them:

```
git add main.py products.json
```

Then create a commit:

```
git commit -m "Add simple product data project"
```

11. Beginner-Friendly Pseudocode

```
START
```

```
import the required data module

create function load_data

    TRY
        open the data file
        read the data
        return the data

    EXCEPT file missing
        print a friendly error message
        return an empty value

call load_data

IF data contains records

    FOR each record
        print selected values

ELSE
    print that no data is available

END
```

Then track the project:

```
open terminal inside project folder

initialize Git repository

check Git status

add project files

create first commit
```

12. Suggested Solving Approach: Functional Approach

Use a small function for loading the data.

Conceptually:

```
main program
  ↓
load_data()
  ↓
open file
```

```
↓  
read structured data  
↓  
return data  
↓  
main program prints values
```

A useful separation is:

```
load_data()  
→ responsible for loading  
  
main program  
→ responsible for displaying
```

This makes your program easier to understand and debug.

Avoid creating many functions today.

One main data-loading function is enough.

13. Easy Edge Cases

Edge Case 1: Missing File

Suppose your code expects:

```
products.json
```

but the file is missing.

Instead of crashing, the program should display something similar to:

```
Could not find the data file.
```

Edge Case 2: Empty Data

Imagine the JSON contains:

```
[]
```

Your program should not attempt to print products that do not exist.

It could display:

```
No product data available.
```

For CSV, you may encounter a file containing only the header:

```
name,price
```

Again, there are no actual product records.

14. Common Mistakes to Avoid

- **Using the wrong filename:** `product.json` and `products.json` are different names.
 - **Using the wrong path:** Python may search for the data file in a different folder than you expect.
 - **Forgetting to import the required module:** JSON needs `import json`; CSV normally needs `import csv`.
 - **Reading JSON without parsing it:** Opening a JSON file and using `read()` gives text. `json.load()` converts JSON into Python objects.
 - **Accessing data before checking whether loading succeeded:** If your function returns empty data after an error, check it before looping.
 - **Putting too much code inside try:** For today's exercise, keep the error-handling section focused on data loading.
 - **Forgetting Git staging:** A changed file is not automatically included in a commit. Usually the flow is `git status` → `git add` → `git commit`.
 - **Expecting GitHub commands automatically:** Today, focus on local Git. GitHub is separate from your local repository.
-

15. Quick Self-Check Questions

Question 1

What is the main difference between Git and GitHub?

Think about:

```
local change tracking  
vs  
online repository hosting
```

Question 2

Which Python module would you normally use for this file?

```
{  
    "name": "Keyboard",  
    "price": 1200  
}
```

Question 3

Why can this line fail even if its Python syntax is correct?

```
open("products.json")
```

Question 4

What is the usual Git order?

```
git commit  
git add  
git status
```

Can you put them in the correct workflow order?

Question 5

Why might a data-loading function return an empty value after handling an error?

Think about how that could allow the rest of the program to decide whether there is data to process.

16. Hint Only

Try using a small JSON file such as:

```
[  
  {"name": "Keyboard", "price": 1200},  
  {"name": "Mouse", "price": 600}  
]
```

Create one function responsible for loading it.

Inside that function, think about this structure:

```
try:  
    # open the file  
    # load JSON  
    # return the data  
  
except ...:  
    # print friendly message  
    # return something representing "no data"
```

Then, outside the function:

```
call function  
check whether data exists
```

```
loop through the records  
print name and price
```

Finally, use the Git workflow you learned:

```
git init  
git status  
git add ...  
git commit ...
```

Your goal for **Day 70** is not to build a large program. It is to demonstrate that you understand the complete beginner workflow:

```
Project  
↓  
Structured data  
↓  
Python function  
↓  
Error handling  
↓  
Useful output  
↓  
Git tracking
```

That combination forms a good foundation for larger data-processing projects later.